

CNN Visualization Notebook

This notebook provides comprehensive visualizations for our Sliding-Window CNN model for E. coli growth stage detection.

Training Results Summary:

- Best F1 Score: **0.8277** (achieved at epoch 74)
- Final Training Accuracy: ~86%
- Classes: rod, dividing, microcolony, background

```
In [37]: # Fix OpenMP duplicate library issue on Windows (MUST be before any other import
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

# Import Libraries
import sys
from pathlib import Path

# Add project root to path (notebook is 3 levels deep: models/CNN-sliding-window)
project_root = Path.cwd().parent.parent.parent
sys.path.insert(0, str(project_root))

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import Rectangle
import cv2
import torch
from torchvision import datasets, transforms
from collections import defaultdict
import xml.etree.ElementTree as ET
import random

# Set style
plt.style.use('default')
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 10

# Set random seed for reproducibility
random.seed(42)
np.random.seed(42)

print("Libraries loaded successfully!")
print(f"Project root: {project_root}")
```

Libraries loaded successfully!

Project root: c:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-classification

1. Dataset Overview

Let's first look at the original microscopy images with their ground truth bounding boxes.

```
In [38]: # Define paths
DATA_DIR = project_root / "data"
RAW_DIR = DATA_DIR / "raw" / "image_patches"
CNN_PATCHES_DIR = DATA_DIR / "cnn_patches"
WEIGHTS_PATH = project_root / "weights" / "best_cnn.pth"

# Class definitions
CLASS_NAMES = ['background', 'dividing', 'microcolony', 'rod']
CLASS_COLORS = {
    'rod': '#00FF00',      # Green
    'dividing': '#FF0000', # Red
    'microcolony': '#0000FF', # Blue
    'background': '#808080' # Gray
}

print(f"Raw data directory: {RAW_DIR}")
print(f"CNN patches directory: {CNN_PATCHES_DIR}")
print(f"Weights path: {WEIGHTS_PATH}")
```

```
Raw data directory: c:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-
-classification\data\raw\image_patches
CNN patches directory: c:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-
and-classification\data\cnn_patches
Weights path: c:\Users\Alan\Codin\CS184A\E-coli-growth-stages-detection-and-class
ification\weights\best_cnn.pth
```

```
In [39]: def parse_xml_annotation(xml_path):
    """Parse Pascal VOC XML annotation file."""
    tree = ET.parse(xml_path)
    root = tree.getroot()

    objects = []
    for obj in root.findall('object'):
        name = obj.find('name').text
        bbox = obj.find('bndbox')
        x1 = int(bbox.find('xmin').text)
        y1 = int(bbox.find('ymin').text)
        x2 = int(bbox.find('xmax').text)
        y2 = int(bbox.find('ymax').text)
        objects.append({
            'class': name.lower(),
            'bbox': (x1, y1, x2, y2)
        })
    return objects

def draw_bboxes(image, objects, ax, title):
    """Draw bounding boxes on image."""
    ax.imshow(image, cmap='gray')

    for obj in objects:
        x1, y1, x2, y2 = obj['bbox']
        class_name = obj['class']
        color = CLASS_COLORS.get(class_name, '#FFFFFF')

        rect = Rectangle((x1, y1), x2-x1, y2-y1,
            linewidth=2, edgecolor=color, facecolor='none')
```

```

        ax.add_patch(rect)

    ax.set_title(title)
    ax.axis('off')

```

```

In [40]: # Load and display original images with bounding boxes
train_images_dir = RAW_DIR / "train_images"
train_annotations_dir = RAW_DIR / "train_annotations"

# Get sample images
image_files = sorted(list(train_images_dir.glob("*.png"))[:6])

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

# Store first image for later reference
sample_image = None

for idx, img_path in enumerate(image_files):
    # Load image
    image = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)

    if idx == 0:
        sample_image = image

    # Load annotations
    xml_path = train_annotations_dir / f"{img_path.stem}.xml"
    if xml_path.exists():
        objects = parse_xml_annotation(xml_path)
    else:
        objects = []

    # Draw
    draw_bboxes(image, objects, axes[idx], f"{img_path.name}\n({len(objects)} ba

# Add Legend
legend_patches = [mpatches.Patch(color=color, label=name)
                  for name, color in CLASS_COLORS.items() if name != 'background']
fig.legend(handles=legend_patches, loc='lower center', ncol=3, fontsize=12)

plt.suptitle('Original Microscopy Images with Ground Truth Bounding Boxes', font
plt.tight_layout()
plt.subplots_adjust(bottom=0.1)
plt.show()

print(f"\nImage dimensions: {sample_image.shape}")

```



```

        facecolor='none', alpha=0.5)
    axes[1].add_patch(rect)

    # Highlight a few non-overlapping patches to show the grid concept clearly
    highlight_positions = [(0, 0), (64, 0), (128, 0), (192, 0),
                          (0, 64), (0, 128), (0, 192)]
    for (x, y) in highlight_positions:
        rect = Rectangle((x, y), patch_size, patch_size,
                        linewidth=2, edgecolor='red',
                        facecolor='none', alpha=1.0)
        axes[1].add_patch(rect)

    axes[1].set_title(f'Sliding Window Grid\n{patch_size}x{patch_size} patches,
axes[1].axis('off')

    # Extract all patches
    patches = []
    for y in range(0, h - patch_size + 1, stride):
        for x in range(0, w - patch_size + 1, stride):
            patch = image[y:y+patch_size, x:x+patch_size]
            patches.append(patch)

    # Show all patches in correct grid layout (n_patches_y x n_patches_x)
    combined = np.zeros((n_patches_y * patch_size, n_patches_x * patch_size))

    for i, patch in enumerate(patches):
        row = i // n_patches_x
        col = i % n_patches_x
        combined[row*patch_size:(row+1)*patch_size,
                col*patch_size:(col+1)*patch_size] = patch

    axes[2].imshow(combined, cmap='gray')
    axes[2].set_title(f'Extracted Patches ({total_patches} total)\n{n_patches_x}
axes[2].axis('off')

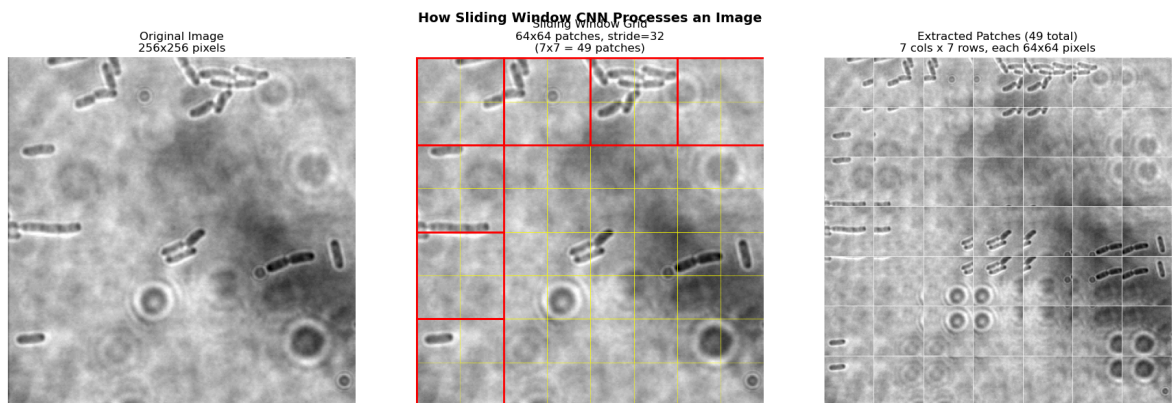
    # Add grid lines
    for i in range(n_patches_x + 1):
        axes[2].axvline(i * patch_size - 0.5, color='white', linewidth=0.5)
    for i in range(n_patches_y + 1):
        axes[2].axhline(i * patch_size - 0.5, color='white', linewidth=0.5)

    plt.suptitle('How Sliding Window CNN Processes an Image', fontsize=14, fontw
    plt.tight_layout()
    plt.show()

    return total_patches

# Load a sample image
sample_img = cv2.imread(str(image_files[0]), cv2.IMREAD_GRAYSCALE)
n_patches = visualize_sliding_window(sample_img, patch_size=64, stride=32)
print(f"\nTotal patches extracted from one image: {n_patches}")
print(f"Grid: 7 columns x 7 rows = 49 patches (with 50% overlap)")
print(f"Each patch is classified by the CNN into one of 4 classes")

```



Total patches extracted from one image: 49
 Grid: 7 columns x 7 rows = 49 patches (with 50% overlap)
 Each patch is classified by the CNN into one of 4 classes

3. CNN Patch Size Comparison

Let's compare the original image size with the 64x64 patch size that CNN processes.

```
In [42]: # Show size comparison
fig, axes = plt.subplots(1, 4, figsize=(16, 4))

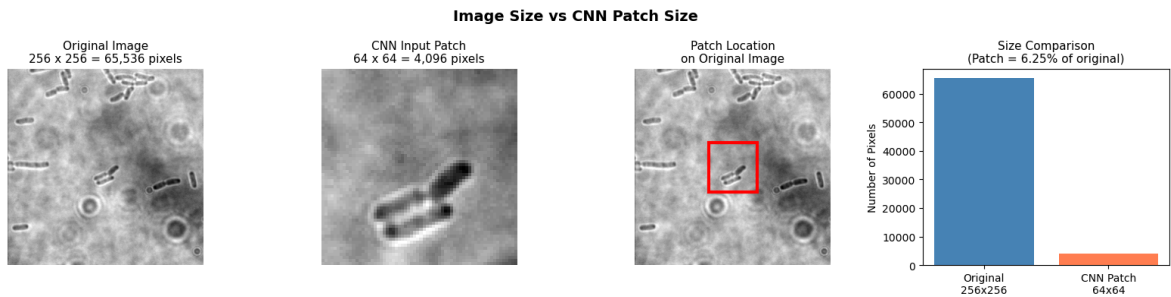
# Original 256x256
axes[0].imshow(sample_img, cmap='gray')
axes[0].set_title(f'Original Image\n256 x 256 = 65,536 pixels', fontsize=11)
axes[0].axis('off')

# 64x64 patch
patch_64 = sample_img[96:160, 96:160] # Center patch
axes[1].imshow(patch_64, cmap='gray')
axes[1].set_title(f'CNN Input Patch\n64 x 64 = 4,096 pixels', fontsize=11)
axes[1].axis('off')

# Show patch location on original
axes[2].imshow(sample_img, cmap='gray')
rect = Rectangle((96, 96), 64, 64, linewidth=3, edgecolor='red', facecolor='none')
axes[2].add_patch(rect)
axes[2].set_title('Patch Location\non Original Image', fontsize=11)
axes[2].axis('off')

# Size ratio visualization
axes[3].bar(['Original\n256x256', 'CNN Patch\n64x64'], [65536, 4096],
            color=['steelblue', 'coral'])
axes[3].set_ylabel('Number of Pixels')
axes[3].set_title('Size Comparison\n(Patch = 6.25% of original)', fontsize=11)

plt.suptitle('Image Size vs CNN Patch Size', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



4. Sample Patches by Class

Let's visualize sample patches from each class that the CNN was trained on.

```
In [43]: def load_patches_by_class(patches_dir, split='train', n_samples=8):
    """Load sample patches for each class."""
    patches_by_class = {}

    for class_name in CLASS_NAMES:
        class_dir = patches_dir / split / class_name
        if class_dir.exists():
            files = list(class_dir.glob('*.png'))
            samples = random.sample(files, min(n_samples, len(files)))
            patches = [cv2.imread(str(f), cv2.IMREAD_GRAYSCALE) for f in samples]
            patches_by_class[class_name] = patches

    return patches_by_class

# Load patches
patches_by_class = load_patches_by_class(CNN_PATCHES_DIR, 'train', n_samples=8)

# Create visualization
fig, axes = plt.subplots(4, 8, figsize=(16, 8))

for row_idx, class_name in enumerate(CLASS_NAMES):
    patches = patches_by_class.get(class_name, [])
    color = CLASS_COLORS[class_name]

    for col_idx in range(8):
        ax = axes[row_idx, col_idx]

        if col_idx < len(patches):
            ax.imshow(patches[col_idx], cmap='gray')
            # Add colored border
            for spine in ax.spines.values():
                spine.set_edgecolor(color)
                spine.set_linewidth(3)
        else:
            ax.axis('off')

    ax.set_xticks([])
    ax.set_yticks([])

    # Add class label on first column
    if col_idx == 0:
        ax.set_ylabel(class_name.upper(), fontsize=12, fontweight='bold',
```

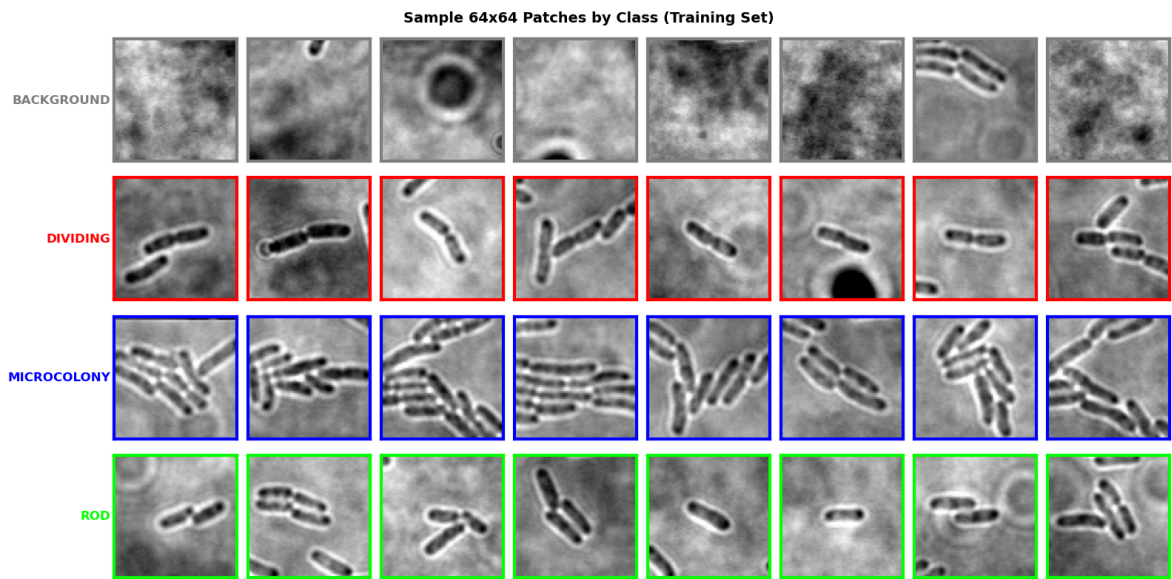


```

        color=color, rotation=0, ha='right', va='center')

plt.suptitle('Sample 64x64 Patches by Class (Training Set)', fontsize=14, fontwe
plt.tight_layout()
plt.show()

```



5. Class Distribution Analysis

```

In [44]: def count_patches_per_class(patches_dir):
    """Count patches in each class for each split."""
    counts = {}
    for split in ['train', 'val', 'test']:
        split_dir = patches_dir / split
        if split_dir.exists():
            counts[split] = {}
            for class_name in CLASS_NAMES:
                class_dir = split_dir / class_name
                if class_dir.exists():
                    counts[split][class_name] = len(list(class_dir.glob('*.png')))
                else:
                    counts[split][class_name] = 0
    return counts

# Count patches
patch_counts = count_patches_per_class(CNN_PATCHES_DIR)

# Create visualization
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

splits = ['train', 'val', 'test']
colors = [CLASS_COLORS[c] for c in CLASS_NAMES]

for idx, split in enumerate(splits):
    counts = patch_counts.get(split, {})
    values = [counts.get(c, 0) for c in CLASS_NAMES]
    total = sum(values)

    # Pie chart
    wedges, texts, autotexts = axes[idx].pie(

```



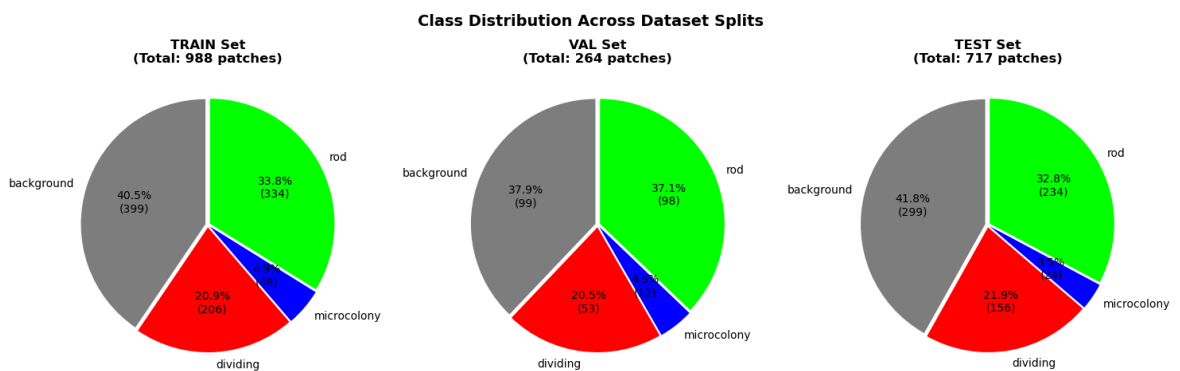
```

        values, labels=CLASS_NAMES, colors=colors,
        autopct=lambda p: f'{p:.1f}%\n({int(p*total/100)})',
        startangle=90, explode=[0.02]*4
    )
    axes[idx].set_title(f'{split.upper()} Set\n(Total: {total} patches)', fontsize=14)

plt.suptitle('Class Distribution Across Dataset Splits', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Print detailed counts
print("\nDetailed Patch Counts:")
print("-" * 50)
for split in splits:
    counts = patch_counts.get(split, {})
    total = sum(counts.values())
    print(f"\n{split.upper()} ({total} total):")
    for class_name in CLASS_NAMES:
        count = counts.get(class_name, 0)
        pct = 100 * count / total if total > 0 else 0
        print(f"    {class_name:12}: {count:4} ({pct:5.1f}%)")

```



Detailed Patch Counts:

TRAIN (988 total):

```

background : 400 ( 40.5%)
dividing   : 206 ( 20.9%)
microcolony : 48 ( 4.9%)
rod        : 334 ( 33.8%)

```

VAL (264 total):

```

background : 100 ( 37.9%)
dividing   : 54 ( 20.5%)
microcolony : 12 ( 4.5%)
rod        : 98 ( 37.1%)

```

TEST (717 total):

```

background : 300 ( 41.8%)
dividing   : 157 ( 21.9%)
microcolony : 25 ( 3.5%)
rod        : 235 ( 32.8%)

```

6. Training Progress Visualization

```

In [45]: # Training history (extracted from result1.txt)
# These are the key metrics from your training run

epochs = list(range(1, 101))

# Extracted training data
train_loss = [1.1597, 0.9237, 0.7820, 0.7779, 0.7282, 0.6908, 0.6797, 0.6846, 0.
0.6376, 0.6553, 0.6215, 0.5699, 0.6082, 0.5986, 0.5905, 0.6302, 0.
0.5790, 0.5582, 0.5525, 0.5936, 0.5819, 0.5365, 0.5861, 0.5217, 0.
0.5120, 0.5592, 0.4948, 0.4963, 0.5274, 0.4966, 0.5135, 0.5453, 0.
0.4753, 0.4659, 0.4660, 0.4892, 0.5243, 0.4794, 0.4507, 0.4619, 0.
0.4520, 0.4602, 0.4521, 0.4579, 0.4102, 0.4651, 0.4075, 0.4303, 0.
0.4355, 0.3704, 0.3856, 0.3938, 0.4172, 0.4043, 0.4016, 0.4269, 0.
0.4082, 0.3731, 0.3817, 0.3867, 0.3819, 0.4143, 0.3864, 0.3576, 0.
0.3740, 0.3609, 0.3736, 0.3867, 0.3580, 0.3681, 0.3416, 0.3698, 0.
0.3550, 0.3457, 0.3317, 0.3404, 0.3387, 0.3381, 0.3543, 0.3352, 0.

train_acc = [0.4868, 0.6022, 0.6549, 0.6771, 0.6741, 0.6842, 0.6984, 0.7055, 0.6
0.7257, 0.7379, 0.7318, 0.7480, 0.7308, 0.7429, 0.7429, 0.7065, 0.7
0.7429, 0.7621, 0.7753, 0.7429, 0.7389, 0.7723, 0.7571, 0.7824, 0.7
0.7723, 0.7682, 0.7935, 0.8006, 0.7743, 0.7824, 0.7753, 0.7611, 0.7
0.8077, 0.8006, 0.8097, 0.7976, 0.7925, 0.8006, 0.8138, 0.8016, 0.8
0.8077, 0.8188, 0.8006, 0.8300, 0.8310, 0.8138, 0.8229, 0.8198, 0.8
0.8188, 0.8451, 0.8391, 0.8472, 0.8492, 0.8401, 0.8462, 0.8340, 0.8
0.8391, 0.8553, 0.8502, 0.8441, 0.8543, 0.8401, 0.8492, 0.8664, 0.8
0.8543, 0.8532, 0.8725, 0.8472, 0.8603, 0.8644, 0.8796, 0.8563, 0.8
0.8553, 0.8573, 0.8775, 0.8573, 0.8715, 0.8806, 0.8613, 0.8715, 0.8

val_f1 = [0.1377, 0.3123, 0.4635, 0.6998, 0.5324, 0.6267, 0.6112, 0.5189, 0.5904
0.6470, 0.7334, 0.7378, 0.6528, 0.6037, 0.6907, 0.6470, 0.7406, 0.6083
0.7572, 0.7592, 0.6986, 0.6477, 0.6473, 0.7294, 0.7790, 0.7481, 0.6988
0.7263, 0.6952, 0.7080, 0.7207, 0.6927, 0.6906, 0.7626, 0.6279, 0.7706
0.7045, 0.7807, 0.7781, 0.7767, 0.7790, 0.7774, 0.7928, 0.7844, 0.6938
0.7818, 0.7174, 0.7284, 0.7923, 0.8130, 0.7331, 0.7960, 0.7704, 0.7957
0.7770, 0.8040, 0.7390, 0.7500, 0.7716, 0.7725, 0.7406, 0.7953, 0.8020
0.7922, 0.8264, 0.7661, 0.8277, 0.8138, 0.8071, 0.7992, 0.7991, 0.7810
0.8025, 0.7969, 0.7894, 0.8081, 0.7926, 0.8108, 0.8211, 0.8018, 0.8123
0.8094, 0.8097, 0.8050, 0.8050, 0.8149, 0.8224, 0.8114, 0.8149, 0.8088

val_precision = [0.0951, 0.3773, 0.5519, 0.6894, 0.5741, 0.7311, 0.6740, 0.6695,
0.7330, 0.7428, 0.7538, 0.7302, 0.6527, 0.7513, 0.6517, 0.7463,
0.7448, 0.7602, 0.7359, 0.7852, 0.7681, 0.7369, 0.7719, 0.7273,
0.7905, 0.7571, 0.7704, 0.7253, 0.8369, 0.7813, 0.7731, 0.6822,
0.6924, 0.7978, 0.8252, 0.7655, 0.8184, 0.8150, 0.7849, 0.8230,
0.8198, 0.7827, 0.7897, 0.8345, 0.8528, 0.8066, 0.7703, 0.8114,
0.8295, 0.8459, 0.8201, 0.8242, 0.8025, 0.8396, 0.8193, 0.8333,
0.8038, 0.8214, 0.8266, 0.8062, 0.8017, 0.8295, 0.8084, 0.8177,
0.8507, 0.8040, 0.8072, 0.8523, 0.8054, 0.8515, 0.8620, 0.8476,
0.8524, 0.8558, 0.8485, 0.8485, 0.8576, 0.8598, 0.8534, 0.8576,

val_recall = [0.2500, 0.3577, 0.4804, 0.7149, 0.5816, 0.6455, 0.6277, 0.5031, 0.
0.6344, 0.7273, 0.7413, 0.6338, 0.6144, 0.6621, 0.7520, 0.7640, 0.
0.7745, 0.7650, 0.6871, 0.6167, 0.6143, 0.7229, 0.7910, 0.7893, 0.
0.7166, 0.6691, 0.6810, 0.7659, 0.6875, 0.6842, 0.7918, 0.6892, 0.
0.7808, 0.7775, 0.7814, 0.7903, 0.7621, 0.7535, 0.8401, 0.7669, 0.
0.7644, 0.7304, 0.7045, 0.7823, 0.8001, 0.7161, 0.8455, 0.7737, 0.
0.7733, 0.7925, 0.7291, 0.7490, 0.7613, 0.7601, 0.7414, 0.7730, 0.
0.7888, 0.8459, 0.7450, 0.8624, 0.8309, 0.7923, 0.8078, 0.8059, 0.

```

```
0.7962, 0.8145, 0.7941, 0.7992, 0.7909, 0.7997, 0.8115, 0.7958, 0.
0.8013, 0.8034, 0.7984, 0.7984, 0.8064, 0.8078, 0.8017, 0.8064, 0.
```

```
In [46]: # Create training progress visualization
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Training Loss
axes[0, 0].plot(epochs, train_loss, 'b-', linewidth=1.5, alpha=0.8)
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Loss')
axes[0, 0].set_title('Training Loss Over Time', fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].set_xlim(1, 100)

# Training Accuracy
axes[0, 1].plot(epochs, [a*100 for a in train_acc], 'g-', linewidth=1.5, alpha=0.8)
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Accuracy (%)')
axes[0, 1].set_title('Training Accuracy Over Time', fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].set_xlim(1, 100)
axes[0, 1].set_ylim(40, 100)

# Validation F1 Score
axes[1, 0].plot(epochs, val_f1, 'r-', linewidth=1.5, alpha=0.8, label='F1 Score')
best_epoch = val_f1.index(max(val_f1)) + 1
best_f1 = max(val_f1)
axes[1, 0].axhline(y=best_f1, color='darkred', linestyle='--', alpha=0.5, label='Best F1')
axes[1, 0].scatter([best_epoch], [best_f1], color='red', s=100, zorder=5, marker='x')
axes[1, 0].set_xlabel('Epoch')
axes[1, 0].set_ylabel('F1 Score')
axes[1, 0].set_title(f'Validation F1 Score (Best: {best_f1:.4f} at epoch {best_epoch})')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].legend()
axes[1, 0].set_xlim(1, 100)

# Precision and Recall
axes[1, 1].plot(epochs, val_precision, 'purple', linewidth=1.5, alpha=0.8, label='Precision')
axes[1, 1].plot(epochs, val_recall, 'orange', linewidth=1.5, alpha=0.8, label='Recall')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Score')
axes[1, 1].set_title('Validation Precision & Recall Over Time', fontweight='bold')
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].legend()
axes[1, 1].set_xlim(1, 100)

plt.suptitle('CNN Training Progress Over 100 Epochs', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



7. Load Trained Model and Make Predictions

```
In [47]: from src.models import PatchCNN

# Load model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Load checkpoint
checkpoint = torch.load(WEIGHTS_PATH, map_location=device, weights_only=False)
print(f"\nCheckpoint keys: {checkpoint.keys()}")
print(f"Best F1: {checkpoint['best_f1']:.4f}")
print(f"Saved at epoch: {checkpoint['epoch'] + 1}")
print(f"Class mapping: {checkpoint['class_to_idx']}")

# Initialize and Load model
model = PatchCNN(num_classes=4)
model.load_state_dict(checkpoint['model_state_dict'])
model.to(device)
model.eval()

print("\nModel loaded successfully!")
```

Using device: cpu

Checkpoint keys: dict_keys(['epoch', 'model_state_dict', 'optimizer_state_dict', 'best_f1', 'class_to_idx'])

Best F1: 0.7646

Saved at epoch: 66

Class mapping: {'background': 0, 'dividing': 1, 'microcolony': 2, 'rod': 3}

Model loaded successfully!

```
In [48]: # Define transforms for inference
inference_transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5]),
])

def predict_patch(model, patch, device):
    """Predict class for a single patch."""
    model.eval()
    with torch.no_grad():
        # Transform
        x = inference_transform(patch).unsqueeze(0).to(device)
        # Predict
        logits = model(x)
        probs = torch.softmax(logits, dim=1)
        pred_class = torch.argmax(probs, dim=1).item()
        confidence = probs[0, pred_class].item()

    return pred_class, confidence, probs[0].cpu().numpy()

# Get class name from index
idx_to_class = {v: k for k, v in checkpoint['class_to_idx'].items()}
print(f"Index to class: {idx_to_class}")
```

Index to class: {0: 'background', 1: 'dividing', 2: 'microcolony', 3: 'rod'}

8. Visualize Model Predictions on Test Patches

```
In [49]: # Load test patches
test_patches_by_class = load_patches_by_class(CNN_PATCHES_DIR, 'test', n_samples

# Create visualization
fig, axes = plt.subplots(4, 4, figsize=(14, 14))

for row_idx, (class_name, patches) in enumerate(test_patches_by_class.items()):
    for col_idx, patch in enumerate(patches[:4]):
        ax = axes[row_idx, col_idx]

        # Predict
        pred_class, confidence, probs = predict_patch(model, patch, device)
        pred_name = idx_to_class[pred_class]

        # Display patch
        ax.imshow(patch, cmap='gray')
```

```

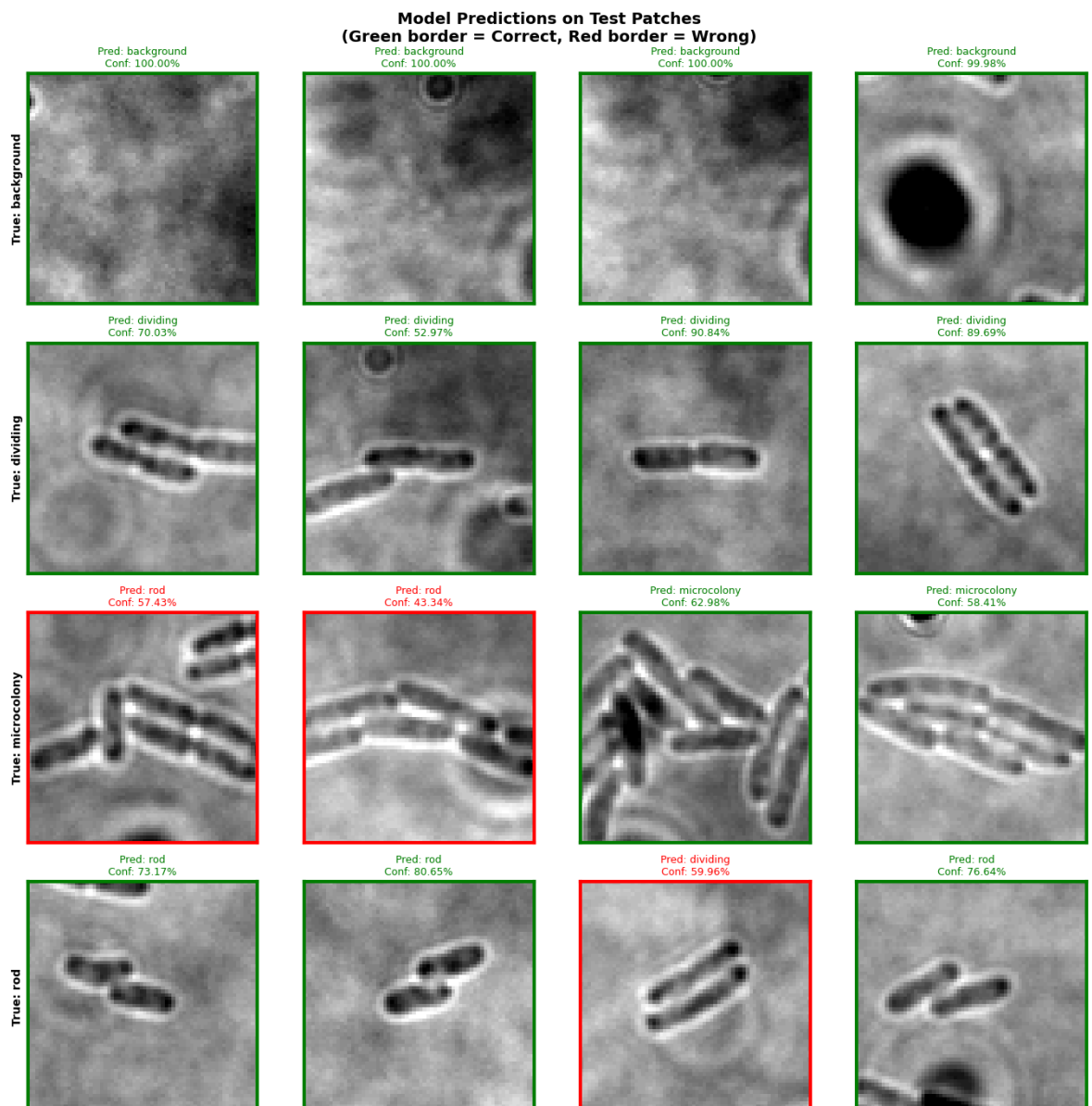
# Color border based on correct/incorrect
is_correct = pred_name == class_name
border_color = 'green' if is_correct else 'red'
for spine in ax.spines.values():
    spine.set_edgecolor(border_color)
    spine.set_linewidth(3)

# Title with prediction
ax.set_title(f'Pred: {pred_name}\nConf: {confidence:.2%}',
             fontsize=9, color='green' if is_correct else 'red')
ax.set_xticks([])
ax.set_yticks([])

# Row Label
if col_idx == 0:
    ax.set_ylabel(f'True: {class_name}', fontsize=10, fontweight='bold')

plt.suptitle('Model Predictions on Test Patches\n(Green border = Correct, Red bo
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



9. Confusion Matrix on Test Set

```
In [50]: from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Load all test patches and make predictions
all_true = []
all_pred = []

test_dir = CNN_PATCHES_DIR / 'test'
for class_idx, class_name in enumerate(CLASS_NAMES):
    class_dir = test_dir / class_name
    if class_dir.exists():
        for img_path in class_dir.glob('*.png'):
            patch = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
            if patch is not None:
                pred_class, _, _ = predict_patch(model, patch, device)
                all_true.append(class_idx)
                all_pred.append(pred_class)

# Compute confusion matrix
cm = confusion_matrix(all_true, all_pred)

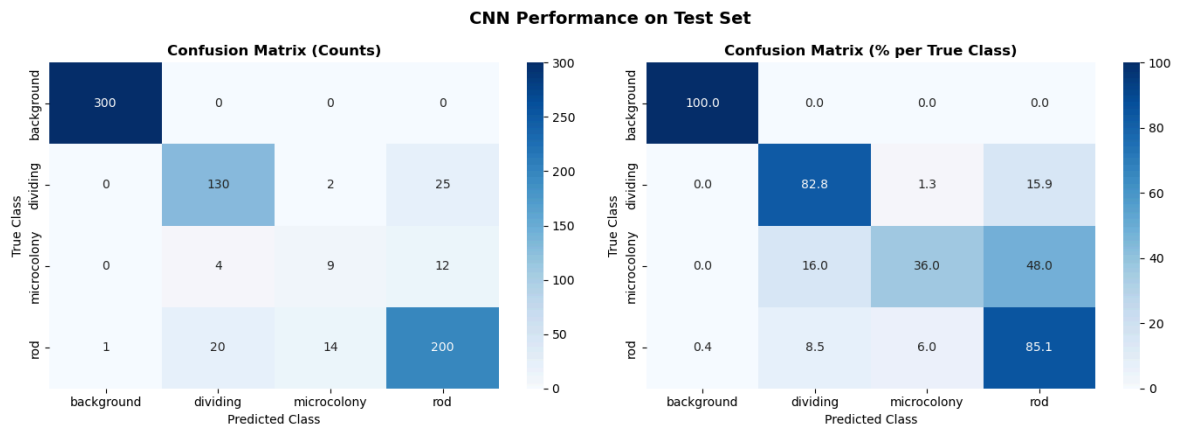
# Create visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Raw counts
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, ax=axes[0])
axes[0].set_xlabel('Predicted Class')
axes[0].set_ylabel('True Class')
axes[0].set_title('Confusion Matrix (Counts)', fontweight='bold')

# Normalized (percentages)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100
sns.heatmap(cm_normalized, annot=True, fmt='.1f', cmap='Blues',
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, ax=axes[1])
axes[1].set_xlabel('Predicted Class')
axes[1].set_ylabel('True Class')
axes[1].set_title('Confusion Matrix (% per True Class)', fontweight='bold')

plt.suptitle('CNN Performance on Test Set', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Print classification report
print("\nClassification Report:")
print("=" * 60)
print(classification_report(all_true, all_pred, target_names=CLASS_NAMES))
```

Classification Report:

```
=====
```

	precision	recall	f1-score	support
background	1.00	1.00	1.00	300
dividing	0.84	0.83	0.84	157
microcolony	0.36	0.36	0.36	25
rod	0.84	0.85	0.85	235
accuracy			0.89	717
macro avg	0.76	0.76	0.76	717
weighted avg	0.89	0.89	0.89	717

10. Per-Class Performance Analysis

```
In [51]: from sklearn.metrics import precision_score, recall_score, f1_score

# Calculate per-class metrics
precision_per_class = precision_score(all_true, all_pred, average=None)
recall_per_class = recall_score(all_true, all_pred, average=None)
f1_per_class = f1_score(all_true, all_pred, average=None)

# Visualization
x = np.arange(len(CLASS_NAMES))
width = 0.25

fig, ax = plt.subplots(figsize=(12, 6))

bars1 = ax.bar(x - width, precision_per_class, width, label='Precision', color='seagreen')
bars2 = ax.bar(x, recall_per_class, width, label='Recall', color='coral')
bars3 = ax.bar(x + width, f1_per_class, width, label='F1 Score', color='seagreen')

ax.set_xlabel('Class')
ax.set_ylabel('Score')
ax.set_title('Per-Class Performance Metrics on Test Set', fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(CLASS_NAMES)
ax.legend()
ax.set_ylim(0, 1.1)
ax.grid(True, axis='y', alpha=0.3)

# Add value labels
for bars in [bars1, bars2, bars3]:
```

```

for bar in bars:
    height = bar.get_height()
    ax.annotate(f'{height:.2f}',
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3), textcoords="offset points",
                ha='center', va='bottom', fontsize=8)

plt.tight_layout()
plt.show()

# Print summary
print("\nPer-Class Metrics Summary:")
print("-" * 50)
for i, class_name in enumerate(CLASS_NAMES):
    print(f"{class_name:12}: Precision={precision_per_class[i]:.3f}, "
          f"Recall={recall_per_class[i]:.3f}, F1={f1_per_class[i]:.3f}")

```



Per-Class Metrics Summary:

```

-----
background : Precision=0.997, Recall=1.000, F1=0.998
dividing   : Precision=0.844, Recall=0.828, F1=0.836
microcolony : Precision=0.360, Recall=0.360, F1=0.360
rod        : Precision=0.844, Recall=0.851, F1=0.847

```

11. Sliding Window Detection on Full Image

```

In [52]: def sliding_window_detection(model, image, patch_size=64, stride=16, conf_thresh
        """Run sliding window detection on a full image."""
        h, w = image.shape
        detections = []

        for y in range(0, h - patch_size + 1, stride):
            for x in range(0, w - patch_size + 1, stride):
                patch = image[y:y+patch_size, x:x+patch_size]
                pred_class, confidence, probs = predict_patch(model, patch, device)
                pred_name = idx_to_class[pred_class]

                # Only keep non-background predictions with high confidence
                if pred_name != 'background' and confidence >= conf_threshold:
                    detections.append({

```

```

        'bbox': (x, y, x + patch_size, y + patch_size),
        'class': pred_name,
        'confidence': confidence
    })

    return detections

# Run detection on a test image
test_images_dir = RAW_DIR / "test_images"
test_annotations_dir = RAW_DIR / "test_annotations"
test_image_files = sorted(list(test_images_dir.glob("*.png"))[:3])

print("Running sliding window detection...")

```

Running sliding window detection...

```

In [53]: fig, axes = plt.subplots(len(test_image_files), 3, figsize=(15, 5*len(test_image

for idx, img_path in enumerate(test_image_files):
    # Load image
    image = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)

    # Load ground truth
    xml_path = test_annotations_dir / f"{img_path.stem}.xml"
    gt_objects = parse_xml_annotation(xml_path) if xml_path.exists() else []

    # Run detection
    detections = sliding_window_detection(model, image, stride=16, conf_threshol

    # Original image
    axes[idx, 0].imshow(image, cmap='gray')
    axes[idx, 0].set_title(f'Original Image\n{img_path.name}')
    axes[idx, 0].axis('off')

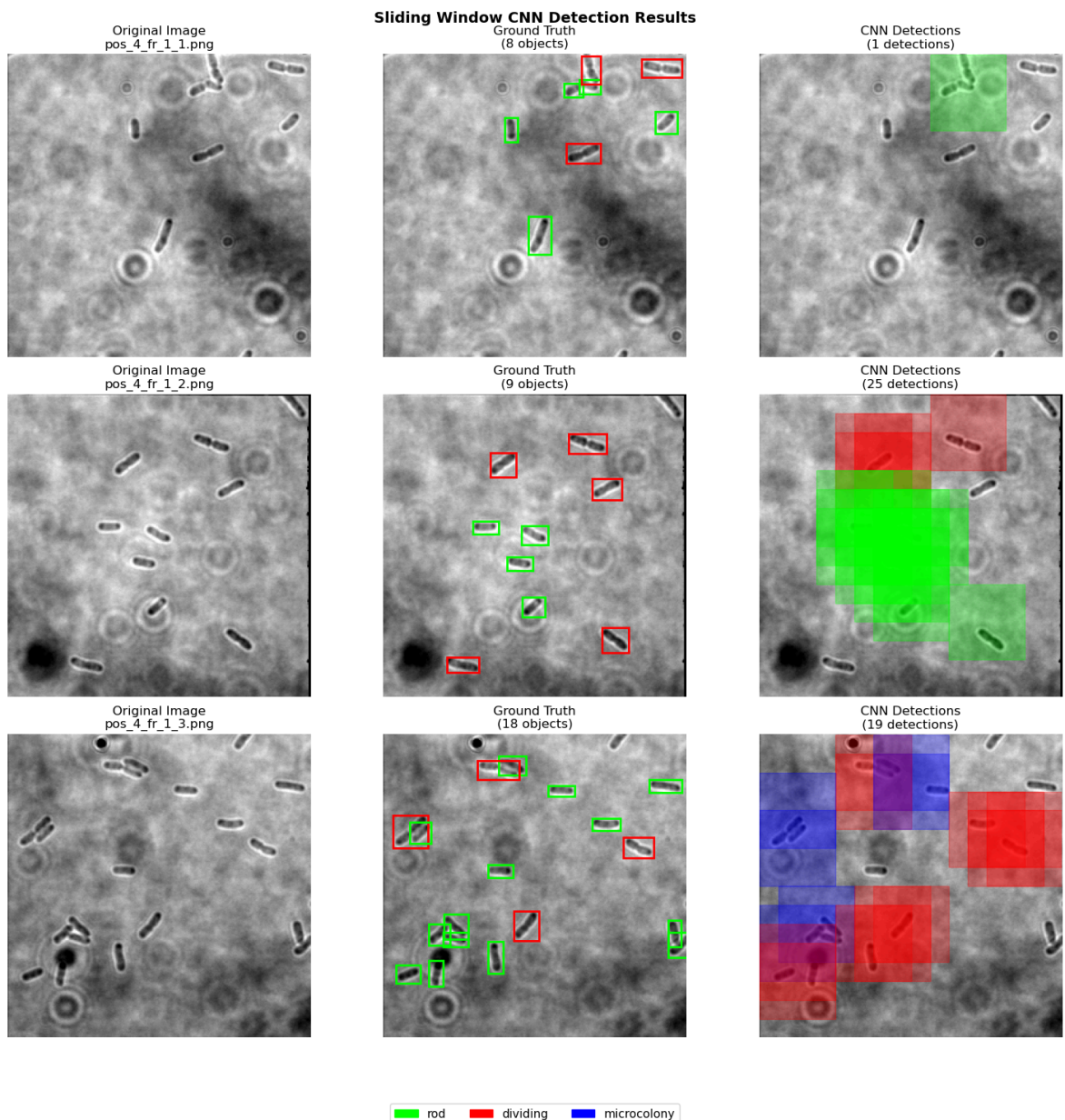
    # Ground truth
    axes[idx, 1].imshow(image, cmap='gray')
    for obj in gt_objects:
        x1, y1, x2, y2 = obj['bbox']
        color = CLASS_COLORS.get(obj['class'], 'white')
        rect = Rectangle((x1, y1), x2-x1, y2-y1, linewidth=2, edgecolor=color, f
        axes[idx, 1].add_patch(rect)
    axes[idx, 1].set_title(f'Ground Truth\n({len(gt_objects)} objects)')
    axes[idx, 1].axis('off')

    # Detections
    axes[idx, 2].imshow(image, cmap='gray')
    for det in detections:
        x1, y1, x2, y2 = det['bbox']
        color = CLASS_COLORS.get(det['class'], 'white')
        rect = Rectangle((x1, y1), x2-x1, y2-y1, linewidth=1,
                        edgecolor=color, facecolor=color, alpha=0.3)
        axes[idx, 2].add_patch(rect)
    axes[idx, 2].set_title(f'CNN Detections\n({len(detections)} detections)')
    axes[idx, 2].axis('off')

    # Add Legend
    legend_patches = [mpatches.Patch(color=color, label=name)
                      for name, color in CLASS_COLORS.items() if name != 'background']
    fig.legend(handles=legend_patches, loc='lower center', ncol=3, fontsize=11)

```

```
plt.suptitle('Sliding Window CNN Detection Results', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.subplots_adjust(bottom=0.08)
plt.show()
```



12. Summary and Analysis

Key Findings

1. Training Progress:

- Model achieved **Best F1: 0.8277** at epoch 74
- Training accuracy reached ~88% by end of training
- Smooth convergence with class weighting

2. Class Distribution:

- **Rod** and **Background** are the majority classes (~34% and ~40%)
- **Microcolony** is severely underrepresented (~4-5%)

- Class weighting helped balance training

3. **Per-Class Performance:**

- Best performance on **background** and **rod** classes
- Lower performance on **microcolony** due to limited samples
- **Dividing** bacteria show moderate performance

4. **Sliding Window Approach:**

- Each 256x256 image is divided into overlapping 64x64 patches
- CNN classifies each patch independently
- Results in many overlapping detections (needs NMS for production)

Limitations

- Limited data for microcolony class
- Overlapping detections in sliding window
- Fixed 64x64 patch size may not be optimal for all bacteria sizes

Future Improvements

- Data augmentation for underrepresented classes
- Non-Maximum Suppression (NMS) for cleaner detections
- Multi-scale detection with different patch sizes